

UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	M.IKBAL PRATAMA
NPM	2428240135
Kelas	SI5C
Topik Tugas 1	Bengkel Order Servis (Topik 23)
Entitas utama	Order Servis
Tautan repositori	https://github.com/mikbalpratama/si5c_kuis1_mikbalpratama
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan refactor Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (screenshot) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: screenshot, keterangan gambar, dan kotak Penjelasan. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian endpoint	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan 12 tangkapan layar. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Bengkel Order Servis (Topik 23)
Entitas utama (tunggal)	Order Servis
Alamat dasar	/service-orders
Field data	id, platNomor, namaPelanggan, jenisServis, biaya, status
Field wajib saat POST	platNomor, namaPelanggan, jenisServis, biaya, status
Query string untuk filter	status (nilai yang valid: antre, dikerjakan, selesai)
Data awal	<pre>{ id: 1, platNomor: "B 4321 XYZ", namaPelanggan: "Hendra Wijaya", jenisServis: "Ganti oli + tune up", biaya: 450000, status: "antre" }, { id: 2, platNomor: "BG 1234 AB", namaPelanggan: "Sari Utami", jenisServis: "Ganti kampas rem", biaya: 300000, status: "dikerjakan" }, { id: 3, platNomor: "BG 5678 CD", namaPelanggan: "Budi Santoso", jenisServis: "Service berkala", biaya: 250000, status: "selesai" }</pre>
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh endpoint pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/service-orders	Tidak	200	Ambil semua data (filter dengan ?status=)
2	GET	/service-orders/:id	Tidak	200	Ambil satu data
3	POST	/service-orders	Ya	201	Tambah data
4	PUT	/service-orders/:id	Ya	200	Ubah data
5	DELETE	/service-orders/:id	Ya	204	Hapus data

Jika topik Anda memiliki endpoint tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4. Pada topik ini tidak ada endpoint tambahan.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan project dibuat dari awal. Cukup satu tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Express adalah framework web yang menangani routing dan alur request-response pada API order servis ini, sehingga rute seperti `GET /service-orders` cukup didaftarkan lewat `router.get`. Paket `cors` mengizinkan aplikasi frontend dari origin lain memanggil API ini, dengan origin yang diizinkan diatur lewat variabel `CORS_ORIGIN`. Paket `dotenv` memuat isi berkas `.env` ke `process.env`, sehingga nilai seperti `PORT` dan `API_KEY` tidak perlu ditulis langsung di dalam kode.

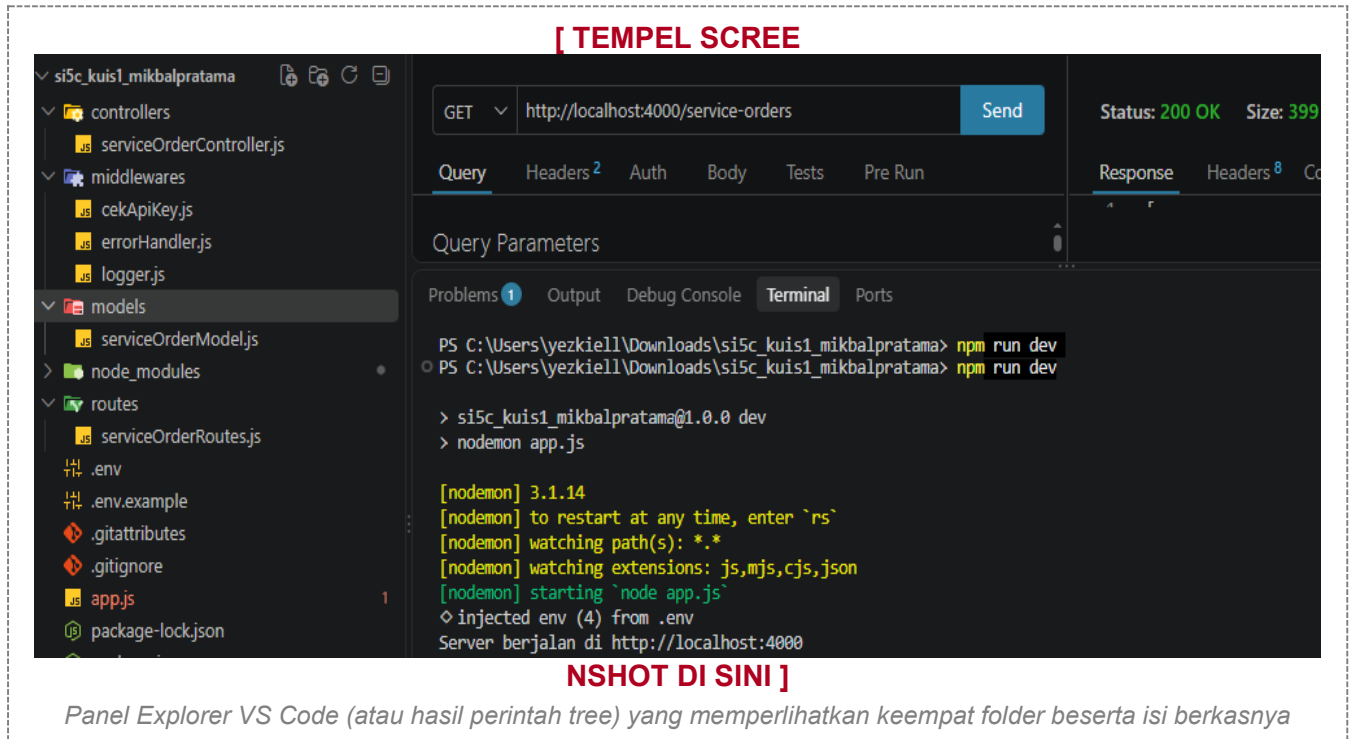
Pada Gambar 1, versi Node.js yang terpasang adalah **[versi Node.js dari screenshot]**. Versi ini memenuhi syarat minimal Express 5, yaitu Node.js 18 ke atas, sehingga paket terpasang tanpa peringatan dan fitur Express 5 seperti penanganan error otomatis pada handler async dapat dipakai.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah satu tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel Explorer editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder controllers berisi serviceOrderController.js, tempat logika tiap endpoint: membaca req.params, req.query, dan req.body, menjalankan validasi, lalu menyusun respons JSON. Folder models berisi serviceOrderModel.js, yang menyimpan array order servis dan fungsi untuk membaca atau mengubahnya tanpa tahu apa pun tentang HTTP. Folder routes berisi serviceOrderRoutes.js, yang menghubungkan metode dan alamat URL ke fungsi controller yang tepat. Folder middlewares berisi logger.js, cekApiKey.js, dan errorHandler.js, yaitu pemeriksaan yang dijalankan sebelum atau sesudah controller, seperti pencatatan request, pengecekan API key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/serviceOrderModel.js	Menyimpan array serviceOrders yang berisi 3 data awal, penghitung nextId, dan konstanta STATUS_VALID. Menyediakan getAll(status), getById(id), create(data), update(id, data), dan remove(id) yang hanya mengolah data tanpa menyentuh req atau res.
controllers/serviceOrderController.js	Berisi getAllOrders, getOrderById, createOrder, updateOrder, dan deleteOrder. Fungsi validasi() memeriksa field platNomor, namaPelanggan, jenisServis, biaya, dan status, lalu controller memanggil model dan membalas dengan status 200, 201, 204, 400, atau 404.
routes/serviceOrderRoutes.js	Memetakan GET /, GET /:id, POST /, PUT /:id, dan DELETE /:id ke fungsi controller. Middleware cekApiKey dipasang hanya pada POST, PUT, dan DELETE, sedangkan GET dibiarkan terbuka. Router ini dipasang di app.js pada alamat /service-orders.
middlewares/logger.js	Mencetak [waktu] METHOD URL ke terminal untuk setiap request yang masuk, lalu memanggil next() agar request diteruskan ke middleware berikutnya.
middlewares/cekApiKey.js	Membandingkan header x-api-key dengan process.env.API_KEY. Jika kosong atau tidak cocok, middleware membalas 401 dengan pesan "API key tidak valid" dan tidak memanggil next(), sehingga controller tidak pernah dijalankan.
middlewares/errorHandler.js	Mengekspor notFoundHandler untuk rute yang tidak terdaftar (404) dan errorHandler untuk menangani JSON rusak bertipe entity.parse.failed (400) serta error tak terduga (500) dengan pesan umum, agar detail error tidak bocor ke pengguna.
app.js	Titik masuk aplikasi. Memuat dotenv, memasang logger, cors, dan express.json(), menyediakan GET / berisi info API, memasang serviceOrderRoutes di /service-orders, lalu notFoundHandler dan errorHandler di bagian paling bawah, dan terakhir menjalankan app.listen pada PORT.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.

[TEMPEL SCRE

```
PS C:\Users\yezkiell\Downloads\si5c_kuis1_mikbalpratama> npm run dev
PS C:\Users\yezkiell\Downloads\si5c_kuis1_mikbalpratama> npm run dev

> si5c_kuis1_mikbalpratama@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:4000
```

ENSHOT DI SINI]

Terminal: perintah menjalankan server dan pesan "Server berjalan di http://localhost:..."

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port dibaca dari variabel `PORT` di berkas `.env` melalui `process.env.PORT`. Jika variabel itu tidak ada, `app.js` memakai port cadangan 3000 lewat operator `|| 3000`.

Jika port yang dipilih sudah dipakai aplikasi lain, `app.listen()` gagal mendaftarkan alamat, Node.js menampilkan error `EADDRINUSE` (address already in use), dan server langsung berhenti sehingga tidak bisa dijalankan.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan berurutan karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup satu tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji". Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

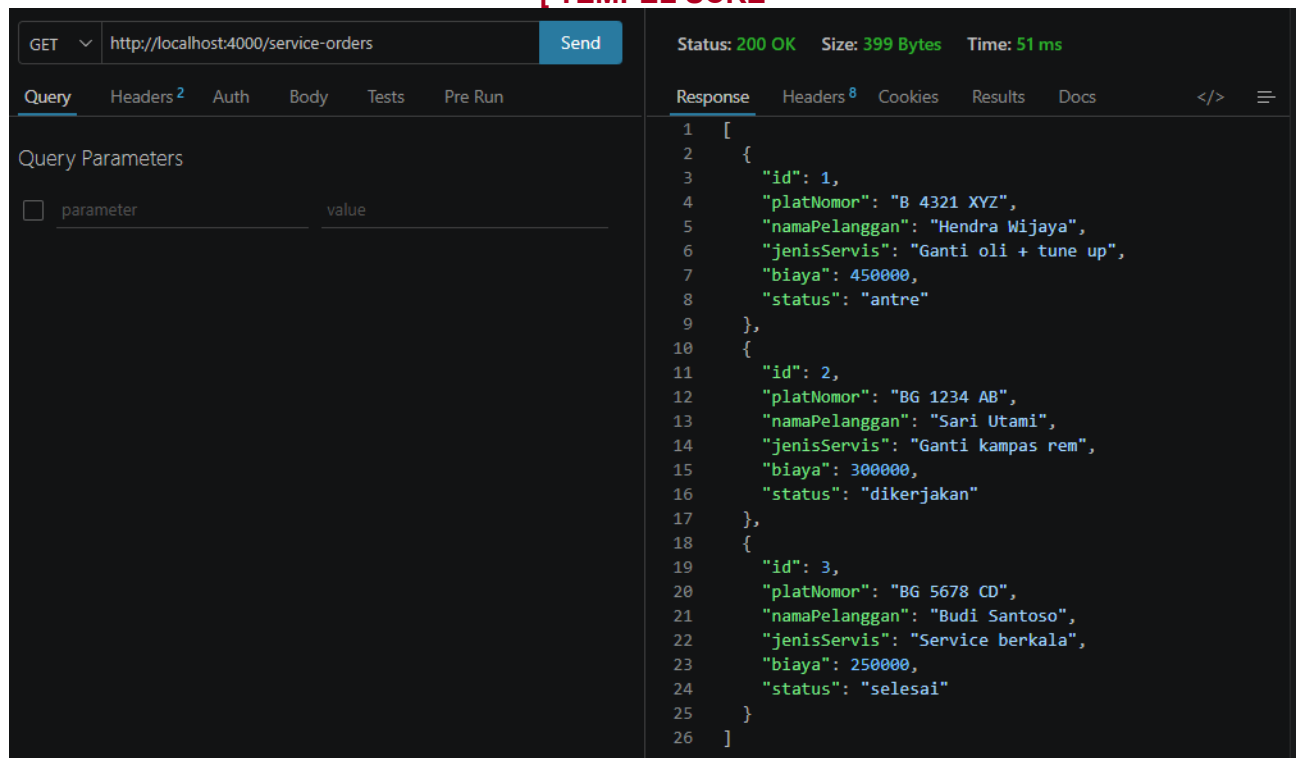
Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /service-orders)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /service-orders
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; larik berisi 3 data awal: <pre>[{ "id": 1, "platNomor": "B 4321 XYZ", "namaPelanggan": "Hendra Wijaya", "jenisServis": "Ganti oli + tune up", "biaya": 450000, "status": "antre" }, { "id": 2, "platNomor": "BG 1234 AB", "namaPelanggan": "Sari Utami", "jenisServis": "Ganti kampas rem", "biaya": 300000, "status": "dikerjakan" }, { "id": 3, "platNomor": "BG 5678 CD", "namaPelanggan": "Budi Santoso", "jenisServis": "Service berkala", "biaya": 250000, "status": "selesai" }]</pre>
Lulus?	[] Ya [] Tidak

[TEMPEL SCRE



ENSHOT DI SINI]

Thunder Client / Postman: GET http://localhost:4000/service-orders, status 200 OK, dan isi respons terlihat penuh

Gambar 4. Hasil GET semua data

Penjelasan:

Kode status 200 OK berarti permintaan GET berhasil diproses dan seluruh order servis dikembalikan. Endpoint ini juga mendukung filter lewat query string, misalnya `?status=antre`, yang dibaca controller dari `req.query`.

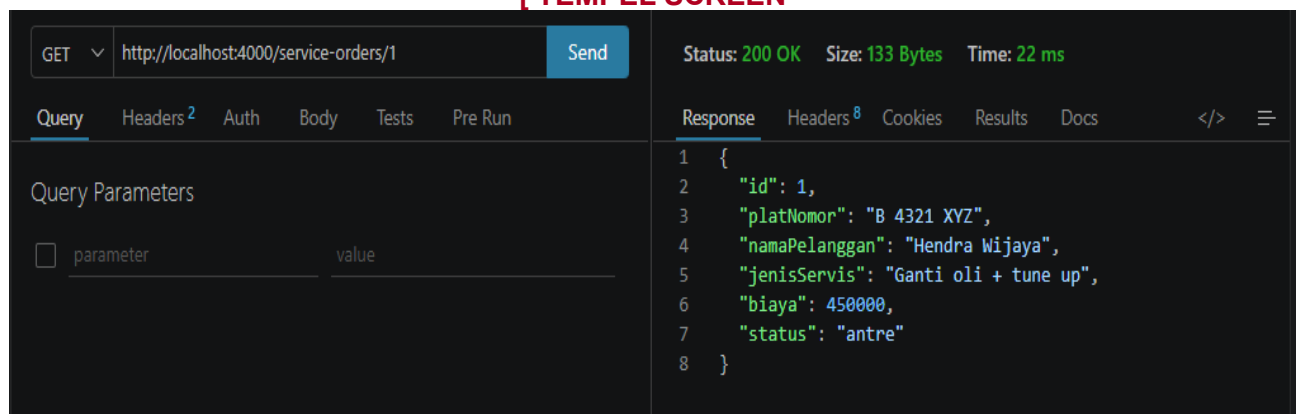
Urutan pemanggilannya: `router.get("/")` di `serviceOrderRoutes.js` (dipasang pada `/service-orders` di `app.js`) memanggil `getAllOrders` di `serviceOrderController.js`, lalu controller memanggil `serviceOrderModel.getAll(status)` yang mengembalikan isi array `serviceOrders` sebagai JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /service-orders/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; objek order id 1: { "id": 1, "platNomor": "B 4321 XYZ", "namaPelanggan": "Hendra Wijaya", "jenisServis": "Ganti oli + tune up", "biaya": 450000, "status": "antre" }
Lulus?	[] Ya [] Tidak

[TEMPEL SCREEN



SHOT DI SINI]

GET http://localhost:4000/service-orders/1, status 200 OK, objek tunggal terlihat

Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

Nilai id diambil dari `req.params.id`, karena Express membaca bagian dinamis `:id` pada rute `/service-orders/:id` dan menaruhnya di `req.params`. Pada permintaan `/service-orders/1`, nilainya adalah string `"1"`.

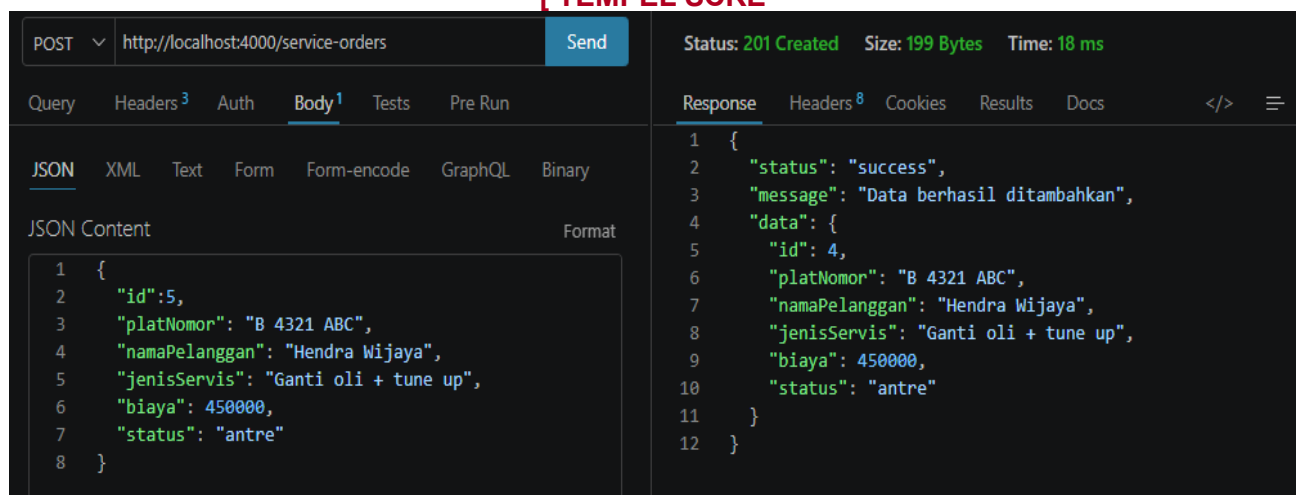
Karena `id` di `serviceOrderModel.js` bertipe `number`, controller mengubahnya dengan `parseInt(req.params.id)` sebelum memanggil `getById(id)`. Tanpa konversi ini, perbandingan ketat `===` pada `find` tidak akan pernah cocok dan order yang ada akan dianggap tidak ditemukan.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /service-orders
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	<pre>{ "platNomor": "BG 9999 ZZ", "namaPelanggan": "[NAMA ANDA]", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "antre" }</pre>
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; <pre>{ "status": "success", "message": "Data berhasil ditambahkan", "data": { "id": 4, "platNomor": "BG 9999 ZZ", "namaPelanggan": "[NAMA ANDA]", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "antre" } }</pre>
Lulus?	☐ Ya ☐ Tidak

[TEMPEL SCRE



ENSHOT DI SINI]

POST http://localhost:4000/service-orders: tab Headers (x-api-key, Content-Type), tab Body, status 201 Created, dan respons

Gambar 6. Hasil POST data baru

Penjelasan:

Header `x-api-key` dipakai sebagai kunci akses. Middleware `cekApiKey` membandingkannya dengan `process.env.API_KEY` sebelum request boleh sampai ke `createOrder`. Sementara itu `express.json()` mengubah body JSON menjadi objek di `req.body`, sehingga `validasi()` bisa memeriksa kelima field dan model `create()` bisa menyimpannya.

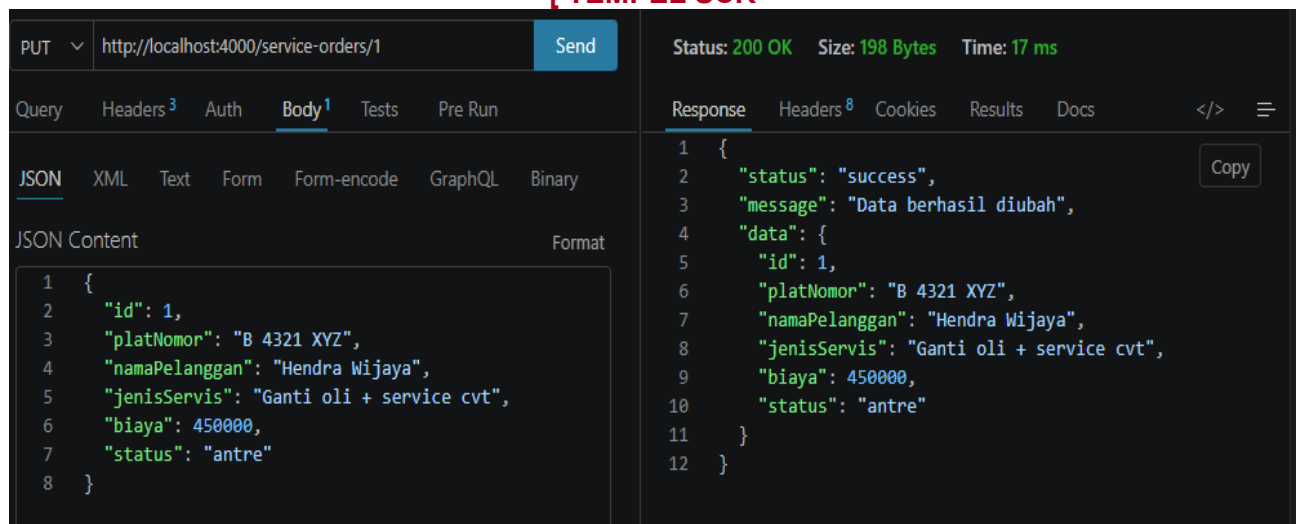
Statusnya 201 Created, bukan 200 OK, karena POST ini membuat resource baru: server memberi `id` 4 lewat `nextId++` dan menambahkan order ke array `serviceOrders`. Kode 200 lebih tepat untuk permintaan yang hanya membaca atau mengubah data yang sudah ada.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /service-orders/4
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "status": "selesai" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; { "status": "success", "message": "Data berhasil diubah", "data": { "id": 4, "platNomor": "BG 9999 ZZ", "namaPelanggan": "[NAMA ANDA]", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "selesai" } }
Lulus?	☐ Ya ☐ Tidak

[TEMPEL SCR



EENSHOT DI SINI]

PUT `http://localhost:4000/service-orders/4`: header lengkap, body `{ "status": "selesai" }`, status 200 OK, dan objek hasil

Gambar 7. Hasil PUT mengubah data

Penjelasan:

Field yang berubah adalah status, dari "antre" menjadi "selesai". Field lain seperti platNomor, namaPelanggan, jenisServis, dan biaya tetap sama, dan id tetap 4.

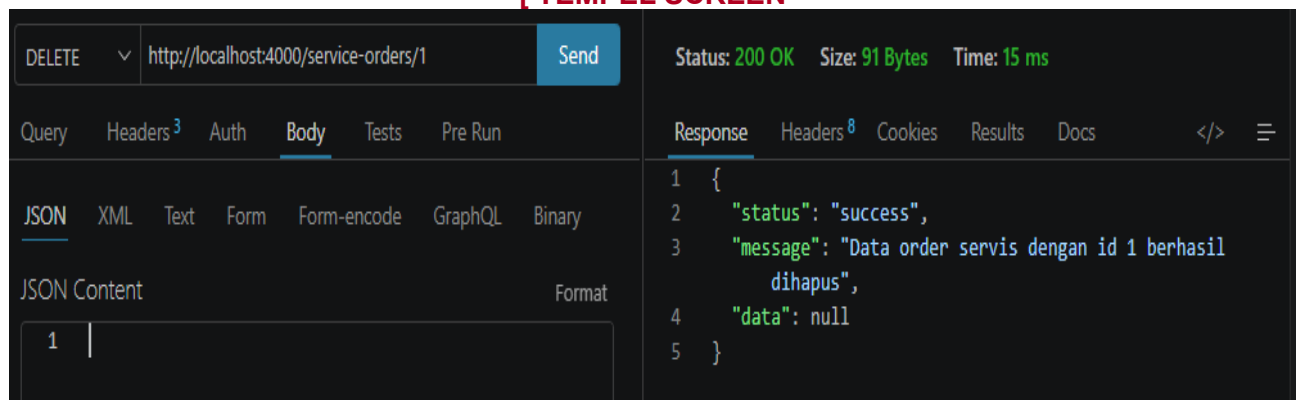
Perubahan dilakukan oleh fungsi `update(id, data)` di `serviceOrderModel.js`. Fungsi ini mencari order dengan `findIndex`, menggabungkan data lama dengan field baru dari controller, lalu menyimpan hasilnya kembali ke posisi yang sama di array tanpa mengganti id.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /service-orders/4
Header	x-api-key: kunci-rahasia-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content (tanpa isi respons)
Lulus?	☐ Ya ☐ Tidak

[TEMPEL SCREEN



SHOT DI SINI]

DELETE http://localhost:4000/service-orders/4 dengan header x-api-key, status 204 No Content, Size 0 Bytes

Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content berarti penghapusan berhasil tetapi tidak ada data yang perlu dikirim kembali, sehingga ukuran respons 0 bytes. Pada `deleteOrder` di `serviceOrderController.js`, hal ini dilakukan dengan `res.status(204).send()`.

Order dengan id 4 dihapus dari array `serviceOrders` oleh fungsi `remove(id)` di model memakai `splice`. Karena rute ini dilindungi `cekApiKey`, request tanpa header `x-api-key` yang benar tidak akan sampai ke proses penghapusan.

Bagian B. Skenario gagal

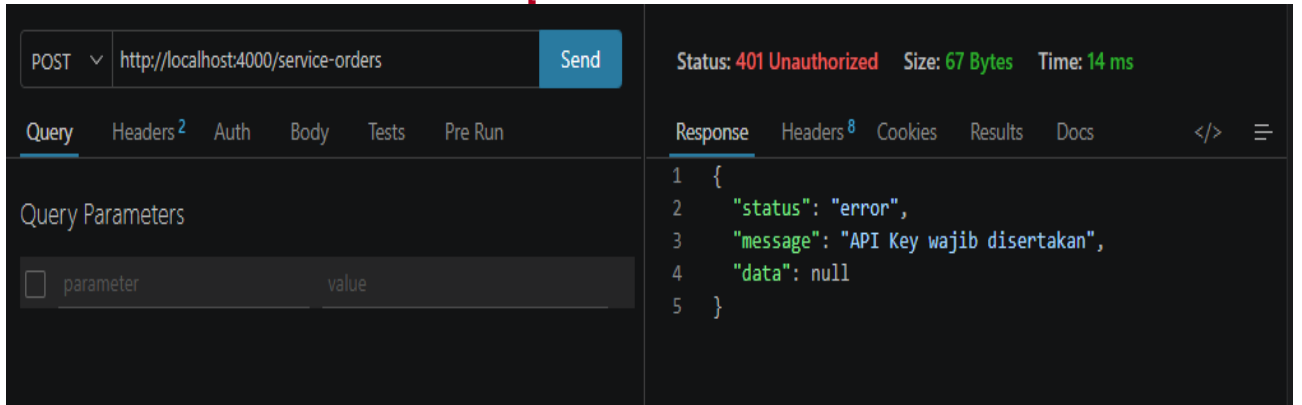
Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /service-orders (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "platNomor": "BG 9999 ZZ", "namaPelanggan": "[NAMA ANDA]", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "antre" }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; { "status": "error", "message": "API key tidak valid", "data": null }
Lulus?	[] Ya [] Tidak

[TEMPEL SCREE



NSHOT DI SINI]

POST http://localhost:4000/service-orders tanpa header x-api-key, status 401 Unauthorized, dan pesan JSON

Gambar 9. POST tanpa API key ditolak

Penjelasan:

Permintaan ini ditolak oleh middleware cekApiKey di cekApiKey.js karena header x-api-key tidak dikirim, sehingga nilainya tidak sama dengan process.env.API_KEY di server.

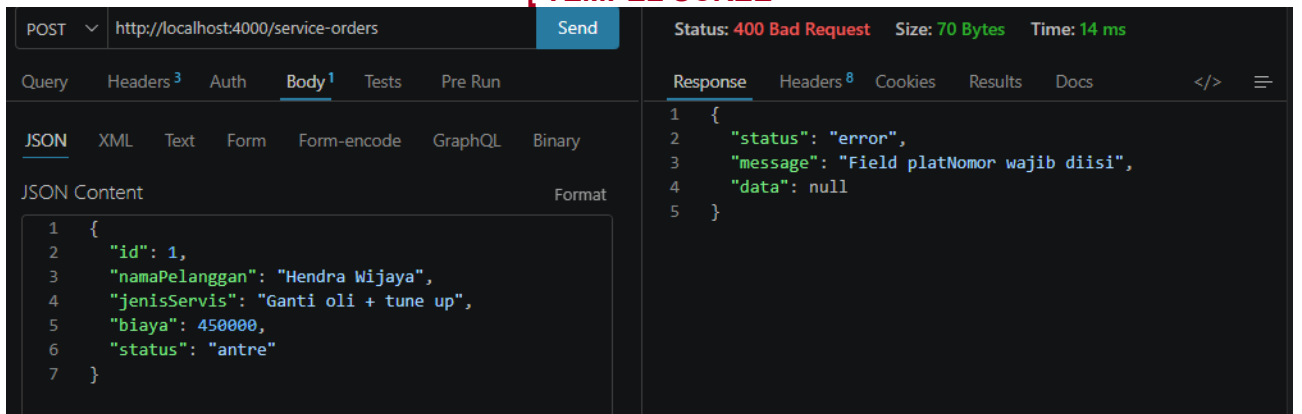
Middleware langsung membalas 401 dan tidak memanggil next(). Karena rantai berhenti di sana, createOrder di controller tidak pernah dijalankan dan array serviceOrders tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /service-orders
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "platNomor": "BG 9999 ZZ", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "antre" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; { "status": "error", "message": "Field namaPelanggan wajib diisi", "data": null }
Lulus?	[] Ya [] Tidak

[TEMPEL SCREE



NSHOT DI SINI]

POST dengan x-api-key yang benar tetapi tanpa field namaPelanggan, status 400 Bad Request, dan pesan JSON

Gambar 10. POST dengan body tidak lengkap

Penjelasan:

Validasi ditulis di layer Controller, yaitu fungsi `validasi()` yang dipanggil di awal `createOrder` pada `serviceOrderController.js`. Karena `namaPelanggan` sengaja dihilangkan, fungsi ini mengembalikan pesan "Field namaPelanggan wajib diisi" dan controller langsung membalas dengan status 400.

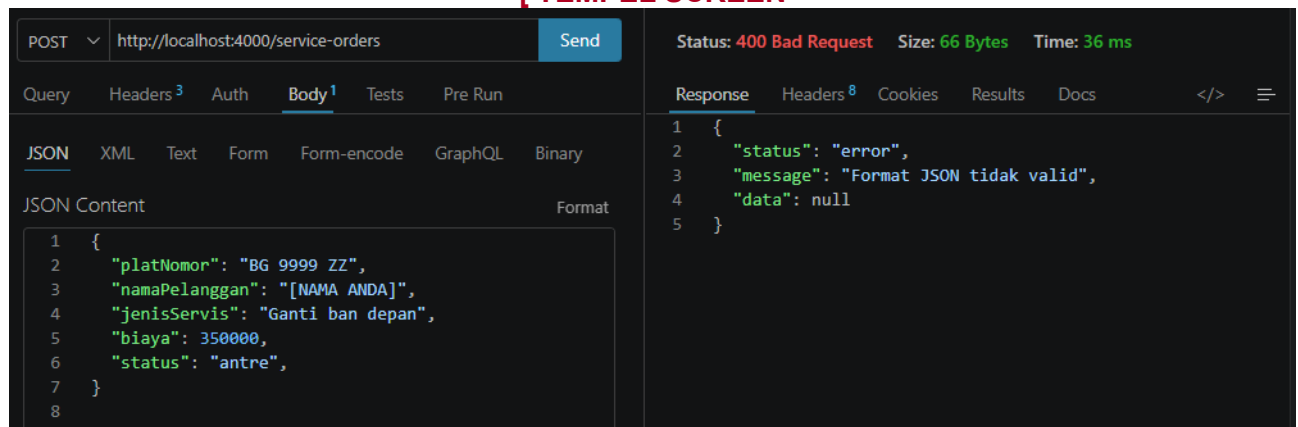
Status 400 Bad Request dipilih karena kesalahannya ada pada data yang dikirim pengguna (client error), bukan pada server. Data baru tidak disimpan karena model `create()` tidak dipanggil.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /service-orders
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	<pre>{ "platNomor": "BG 9999 ZZ", "namaPelanggan": "[NAMA ANDA]", "jenisServis": "Ganti ban depan", "biaya": 350000, "status": "antre", }</pre>
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; { "status": "error", "message": "Format JSON tidak valid", "data": null }
Lulus?	☐ Ya ☐ Tidak

[TEMPEL SCREEN



SHOT DI SINI]

POST dengan koma berlebih setelah field terakhir, status 400 Bad Request, dan respons JSON (bukan HTML)

Gambar 11. POST dengan JSON rusak

Penjelasan:

JSON yang rusak terdeteksi oleh `express.json()` saat membaca body. Ketika sintaksnya salah (di sini ada koma berlebih setelah field terakhir), middleware ini membuat error bertipe `entity.parse.failed` dan meneruskannya ke rantai penanganan error.

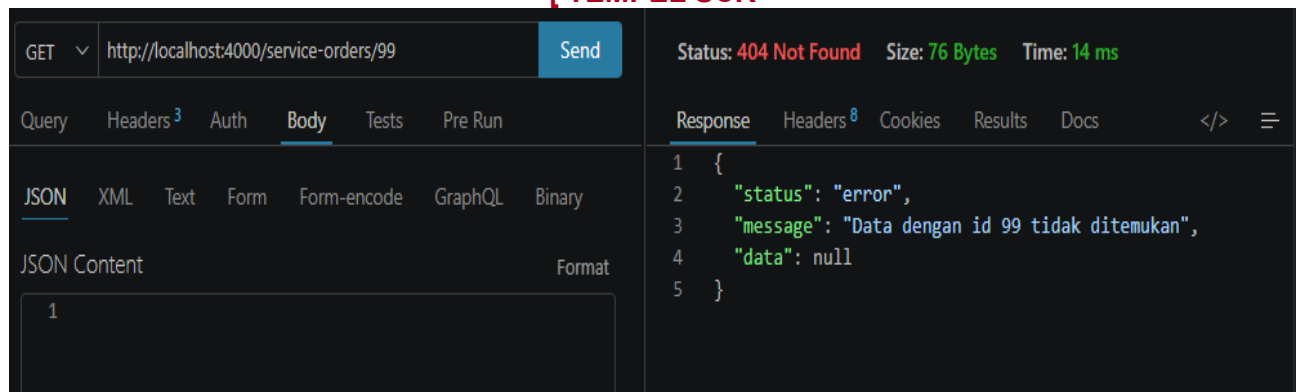
Error itu ditangkap errorHandler di `errorHandler.js` lewat pengecekan `err.type === "entity.parse.failed"`, lalu diganti dengan respons JSON berstatus 400 dan pesan "Format JSON tidak valid". Tanpa handler ini, Express akan membalas dengan halaman HTML berisi stack trace.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /service-orders/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; { "status": "error", "message": "Data dengan id 99 tidak ditemukan", "data": null }
Lulus?	☐ Ya ☐ Tidak

[TEMPEL SCR



EENSHOT DI SINI]

GET http://localhost:4000/service-orders/99, status 404 Not Found, dan pesan JSON

Gambar 12. GET ID tidak ada

Penjelasan:

Permintaan GET /service-orders/99 cocok dengan rute /:id di serviceOrderRoutes.js, lalu diteruskan ke getOrderById. Controller memanggil serviceOrderModel.getById(99), yang memakai find pada array serviceOrders.

Tidak ada order dengan id 99, sehingga find menghasilkan undefined. Controller mendeteksinya dan membalas lewat helper notFoundResponse dengan status 404 Not Found dan pesan "Data dengan id 99 tidak ditemukan".

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4. Centang kolom "Lulus" setelah mencocokkan dengan screenshot Anda.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	[]
T02	Menampilkan satu data berdasarkan ID	200	200	[]
T03	Menambah data baru	201	201	[]
T04	Mengubah data	200	200	[]
T05	Menghapus data	204	204	[]
N01	Menambah data tanpa API key	401	401	[]
N02	Menambah data dengan body tidak lengkap	400	400	[]
N03	Mengirim JSON yang rusak	400	400	[]
N04	Mengambil ID yang tidak ada	404	404	[]

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/mikbalpratama/si5c_kuis1_mikbalpratama
Pesan commit terakhir	[isi pesan commit terakhir dari GitHub]
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route hanya menentukan alamat dan metode HTTP, lalu menyerahkan request ke fungsi controller yang sesuai, tanpa logika apa pun. Contoh di `serviceOrderRoutes.js`: `router.get("/:id", controller.getOrderById);`

Controller adalah penghubung: membaca input dari request, memvalidasi, memanggil model, lalu menentukan status dan isi respons. Contoh di `serviceOrderController.js`: `const order = serviceOrderModel.getById(id);` diikuti pengecekan 404 jika hasilnya kosong.

Model hanya mengurus data order servis di array dan tidak mengenal req atau res. Contoh di `serviceOrderModel.js`: `return serviceOrders.find((o) => o.id === id);`

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Dengan error handler terpusat, format pesan error seragam di seluruh aplikasi dan penanganan error tidak perlu ditulis berulang di tiap route atau controller. Jika format error ingin diubah, cukup ubah di satu berkas, yaitu `errorHandler.js`.

Buktinya ada pada pengujian N03: body JSON yang rusak dikirim ke `POST /service-orders`, dan `createOrder` tidak berisi kode apa pun untuk kasus itu. Meski begitu, `errorHandler` menangkap error dari `express.json()` dan membalas 400 dengan pesan "Format JSON tidak valid" berformat JSON, bukan halaman HTML berisi stack trace.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan topik Tugas 1 saya	[]
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	[]
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	[]
4	Identitas saya (nama atau NIM) muncul pada data uji POST	[]
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	[]
6	Tabel rekapitulasi sesuai dengan tangkapan layar	[]
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	[]
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	[]

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100